SIMATS ENGINEERING

CO3-ASSESSMENT TOOL 1

Experiments

COURSE NAME: Computer Networks

COURSE CODE: CSA0713

COURSE OUTCOME COVERED

CO3: Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions

Assessment Tool Weightage: 40%

Dr. V. Parthipan

Code of Conduct:

I Levin Anish .A(192511223) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

Levin Anish .A

QUESTIONS:4 Total Marks:40

1. Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, and ACK). Demonstrate the role of each flag and how they establish a reliable connection. (BL4) (10)

Aim

To capture packets during a TCP connection setup and identify the three packets involved in the TCP three-way handshake.

Procedure

1. Open Wireshark.
2. Select the active network interface (Wi-Fi or Ethernet).
3. Start packet capture.
4. Open a web browser and visit any website (e.g., www.google.com).
5. Stop the packet capture.
6. Apply the filter **tcp**.
7. Identify the three packets involved in the connection setup.

Explanation

TCP establishes a reliable connection using a three-way handshake. The client first sends a SYN packet to request a connection. The server replies with a SYN-ACK packet, indicating that it has received the request and is ready to communicate. Finally, the client sends an ACK packet, confirming receipt of the server's response. After these three packets are exchanged, the TCP connection is established and data transfer begins.

Observations

During the packet capture, three important packets were observed before any application data was transmitted. The first packet contained the **SYN** flag, indicating that the client wanted to establish a TCP connection with the server. The second packet contained both the **SYN** and **ACK** flags, showing that the server had accepted the request and was ready to communicate. The third packet contained only the **ACK** flag, confirming that the client had received the server's response. After the successful exchange of these three packets, the TCP connection was established and data transfer began.

Role of TCP Flags

The **SYN (Synchronize)** flag is used to initiate a new TCP connection. It allows the client to send its initial sequence number to the server. The **SYN-ACK (Synchronize-Acknowledgment)** flag is sent by the server to acknowledge the client's request and provide its own sequence number. Finally, the **ACK (Acknowledgment)**

Discussion

The TCP three-way handshake is an essential mechanism that establishes a reliable communication channel between two devices. During this process, both the client and the server agree on the initial sequence numbers that will be used for tracking data packets. This synchronization prevents duplicate packets, detects missing packets, and ensures that data is delivered in the correct order. Since both devices confirm their readiness before communication begins, TCP provides reliable and error-free transmission.

Result

The TCP three-way handshake successfully establishes a reliable connection between the client and the server before any data is transmitted.

2. Capture a DNS query using UDP in Wireshark. Compare the UDP header size

with TCP and note the absence of sequence numbers or acknowledgments.

Discuss how this lightweight design impacts speed and reliability. (BL4)

(10)

Aim

To capture a DNS query using UDP in Wireshark, compare the UDP header size with the TCP header, observe the absence of sequence numbers and acknowledgments in UDP, and understand how its lightweight design affects speed and reliability.

Procedure

1. Open Wireshark and select the active network interface (Wi-Fi or Ethernet).
2. Start capturing packets.
3. Apply the display filter **dns** to capture only DNS packets.
4. Open a web browser and visit a website (for example, **www.google.com**) or use the **nslookup** command to generate a DNS query.
5. Wait until the DNS request and response are displayed in Wireshark.
6. Stop the packet capture.
7. Select the DNS query packet and examine the UDP header details.

Observations

The captured DNS packet used the **User Datagram Protocol (UDP)** with **destination port 53**, which is the standard port for DNS communication. The UDP header consisted of only four fields: Source Port, Destination Port, Length, and Checksum, giving it a total header size of **8 bytes**. Unlike TCP, the UDP packet did not contain sequence numbers, acknowledgment numbers, window size, or connection establishment information. The DNS query was transmitted immediately without performing any

handshake, and the server responded with the requested IP address. The entire communication involved only a request and a response, making the process fast and efficient.

Comparison of UDP and TCP

Feature	**UDP**	**TCP**
Header Size	8 Bytes	20 Bytes (Minimum)
Connection Type	Connectionless	Connection-Oriented
Sequence Numbers	Not Present	Present
Acknowledgment Numbers	Not Present	Present
Error Recovery	Not Available	Available
Retransmission	No	Yes
Communication Speed	Faster	Slower
Reliability	Lower	Higher

Discussion

UDP is a lightweight transport layer protocol because it uses a very small header of only **8 bytes**, compared to the minimum **20-byte** header used by TCP. Since UDP does not establish a connection before transmitting data, it avoids the overhead of the TCP three-way handshake. It also does not maintain sequence numbers or acknowledgment numbers, which reduces processing time and improves transmission speed.

Because of this lightweight design, UDP is widely used for applications where quick communication is more important than guaranteed delivery. DNS is one such application because DNS queries and responses are usually small and can be completed with a single request and response. Even if a DNS packet is lost, the client can simply send another request instead of relying on retransmissions.

Result

The DNS query was successfully captured using UDP in Wireshark. The analysis showed that the UDP header is only **8 bytes**, whereas the TCP header is at least **20 bytes**. The absence of sequence numbers and acknowledgment fields makes UDP faster and more efficient for applications like DNS, but it is less reliable than TCP because it does not provide retransmission or error recovery mechanisms.

3. Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark.

Observe how TCP retransmits lost packets while UDP does not. Explain why this

difference makes TCP reliable but slower compared to UDP. (BL4) (10)

Aim

To simulate packet loss using Wireshark and observe how TCP retransmits lost packets while UDP does not. The experiment also aims to understand why TCP is considered reliable but slower compared to UDP.

Procedure

1. Open Wireshark and select the active network interface.
2. Start packet capture.
3. Begin downloading a file or browse a website that uses TCP.
4. Briefly disconnect the network connection (Wi-Fi or Ethernet) for a few seconds.
5. Reconnect the network and allow the download or webpage to continue.
6. Stop the packet capture.
7. Apply the display filter **tcp** and observe retransmitted packets.
8. Repeat the experiment while generating UDP traffic, such as performing a DNS lookup or using another UDP-based application.
9. Compare the behavior of TCP and UDP after packet loss.

Observations

During the TCP communication, packet loss occurred when the network connection was briefly interrupted. Wireshark displayed **TCP Retransmission** and **Duplicate ACK** packets, indicating that TCP detected the missing packets and automatically retransmitted them. Once the lost packets were successfully received, the communication continued without losing any data.

When UDP traffic was observed under similar conditions, the lost packets were not retransmitted. The receiver did not send acknowledgments, and the sender did not attempt to resend the missing packets. As a result, the lost data could not be recovered, and communication continued with the missing information.

Discussion

TCP is a connection-oriented protocol that ensures reliable data transmission through several mechanisms, including sequence numbers, acknowledgments, retransmissions, and flow control. Each transmitted packet is assigned a sequence number, allowing the receiver to verify whether all packets have arrived in the correct order. If a packet is lost during transmission, the receiver either sends duplicate acknowledgments or waits until the sender's retransmission timer expires. The sender then retransmits the missing packet, ensuring that no data is permanently lost.

Although these reliability mechanisms improve data integrity, they also introduce additional overhead. TCP requires acknowledgments for received packets, continuously tracks sequence numbers, and retransmits lost packets whenever necessary. These extra operations consume processing time and network bandwidth, making TCP slower than UDP, especially in networks with high packet loss or latency.

UDP, on the other hand, is a connectionless protocol that does not establish a connection before transmitting data. It does not use sequence numbers, acknowledgments, retransmissions, or flow control. Packets are transmitted independently without verifying whether they have reached the

destination successfully. Because UDP avoids these additional processes, it has very low protocol overhead and provides faster data transmission. However, if packets are lost, corrupted, or received out of order, UDP does not attempt to recover them. This makes UDP suitable for applications such as live video streaming, online gaming, voice communication, and DNS, where speed is more important than perfect reliability.

Therefore, the fundamental difference between TCP and UDP lies in their approach to reliability. TCP guarantees accurate and complete data delivery through retransmission mechanisms, whereas UDP prioritizes speed and efficiency by sacrificing reliability.

Comparison of TCP and UDP During Packet Loss

Feature	**TCP**	**UDP**
Connection Type	Connection-Oriented	Connectionless
Packet Retransmission	Yes	No
Sequence Numbers	Present	Not Present
Acknowledgments	Present	Not Present
Error Recovery	Available	Not Available
Data Reliability	High	Low
Transmission Speed	Slower	Faster
Suitable Applications	File Transfer, Email, Web Browsing	DNS, VoIP, Video Streaming, Online Gaming

Result

The experiment demonstrated that TCP automatically retransmits lost packets and ensures reliable data delivery using acknowledgments and sequence numbers. In contrast, UDP does not retransmit lost packets and therefore provides faster communication with lower overhead but without guaranteeing successful delivery. This confirms that TCP is more reliable but slower, whereas UDP is faster but less reliable.

4. Use Wireshark to capture DNS traffic while resolving a domain name. Identify the

query type (A, AAAA, MX) and the response received. Measure the time taken for

resolution and explain why DNS typically uses UDP.(BL4) (10)

RUBRICS FOR CALCULATION:

Aim:

To capture DNS traffic using Wireshark while resolving a domain name, identify the DNS query type and response, measure the DNS resolution time, and explain why DNS typically uses UDP.

Procedure

1. Open Wireshark and select the active network interface (Wi-Fi/Ethernet).
2. Start packet capturing.
3. Open a web browser and visit a website (e.g., **www.google.com**).
4. After the webpage loads, stop the packet capture.
5. Apply the display filter:
6. Locate the DNS Query and DNS Response packets.
7. Note the query type, response received, and timestamps.
8. Calculate the DNS resolution time using the timestamps.

Observations

Parameter	Observation
Domain Name	www.google.com
Protocol	DNS
Query Type	A (IPv4 Address Record)
Source IP	192.168.1.5 (Example)
Destination IP	8.8.8.8 (DNS Server)
Response Received	IPv4 Address (e.g., 142.250.183.68)
Transport Protocol	UDP
DNS Resolution Time	18.591 ms (Example)

DNS Query Types

A Record

Returns the IPv4 address of the requested domain name.

AAAA Record

Returns the IPv6 address of the requested domain name.

MX Record

Returns the Mail Exchange server information responsible for receiving emails for the domain.

Observed Query Type: A Record

Response Received

The DNS server returned the IPv4 address corresponding to **www.google.com**. This IP address allows the web browser to establish communication with the destination web server.

Example Response:

www.google.com → 142.250.183.68

Calculation of DNS Resolution Time

The DNS resolution time is calculated using the timestamps of the DNS Query and DNS Response packets captured in Wireshark.

Formula

DNS Resolution Time = Response Timestamp − Query Timestamp

Example Calculation

Query Timestamp = **12.450321 s**

Response Timestamp = **12.468912 s**

DNS Resolution Time

= **12.468912 − 12.450321**

= **0.018591 seconds**

= **18.591 milliseconds**

Result

Parameter	Value
DNS Query Type	A Record
Response Received	IPv4 Address
Resolution Time	18.591 ms

Parameter	Value
Transport Protocol	UDP